



Tecnicatura Universitaria en Inteligencia Artificial
Facultad de Ciencias Exactas, Ingeniería y
Agrimensura
Universidad Nacional de Rosario

Trabajo Práctico Final

Cátedra: **PROCESAMIENTO DEL LENGUAJE NATURAL**

2025

Profesores:

- Juan Pablo Manson
- Alan Geary
- Constantino Ferrucci
- Sol Berger Dolores

Alumna:

- Rizzotto, María Camila



Contenido

Resumen.....	3
Introducción	3
Ejercicio 1 – Chatbot con RAG.....	4
Tarea 1 - Creación de la base de datos vectorial	4
Tarea 2: Acceso y filtrado semántico de datos estadísticos	4
Tarea 3: Construcción de una base de datos de grafos y consultas en Cypher	5
Tarea 4: Desarrollo de un clasificador de intención avanzado	6
Tarea 5: Definición del pipeline de recuperación (retrieval)	7
Tarea 6: Integración para generación y conversación	7
Ejercicio 2: Implementación de un agente autónomo ReAct con LangChain	9
Conclusión	11

Resumen

Este trabajo aborda el desarrollo de un asistente conversacional especializado en el juego de mesa Pradera, combinando técnicas de Recuperación Aumentada por Generación (RAG) y un agente autónomo basado en el paradigma ReAct. Los objetivos incluyeron la creación de tres motores de búsqueda—vectorial (híbrida + re-rank), tabular (filtros dinámicos) y de grafos (consultas Cypher)—junto a un clasificador de intención avanzado y herramientas de búsqueda online (Wikipedia y DuckDuckGo). Todo esto fue realizado en un notebook de Colab.

En la fase de implementación, se definieron funciones independientes para cada fuente de datos y un prompt de sistema altamente estructurado que guía al modelo LLM del agente autónomo (Mistral-7B-Instruct vía Together.ai) a razonar paso a paso (Thought → Action → Action Input → Observation → Final Answer). El agente resultante demostró relevancia en las respuestas multilingües, mantenimiento de contexto y gestión adecuada de consultas sin coincidencias. Por otro lado, se compararon dos clasificadores de intención, en donde un modelo basado en LLM superó a otro basado en redes neuronales en base a la métrica Accuracy.

Las conclusiones confirman el cumplimiento íntegro de los objetivos y abren la puerta a futuras mejoras: métricas automáticas de evaluación, memorias a largo plazo, y optimización de la infraestructura para reducir los tiempos de generación de respuesta.

Introducción

En el presente trabajo se desarrolla un chatbot experto sobre el juego de mesa *Pradera*, diseñado con una arquitectura RAG (*Retrieval-Augmented Generation*).

La construcción de este sistema se planteó como una continuación del Trabajo Práctico 1, reutilizando el corpus previamente generado. Dicho corpus se compone de tres fuentes de datos principales:

- (1) Información: textos informativos extraídos de manuales, foros y otros documentos en formato PDF;
- (2) Estadísticas: estadísticas del juego almacenadas en un archivo .csv;
- (3) Relaciones: relaciones estructuradas entre elementos del juego, también en formato .csv.

A lo largo del desarrollo, se abordaron diversas tareas incrementales orientadas a enriquecer la representación del conocimiento, adecuar el corpus a requerimientos semánticos y técnicos de la arquitectura RAG, y garantizar

respuestas precisas, contextualizadas y relevantes por parte del sistema conversacional. Este informe detalla el enfoque metodológico adoptado, las decisiones de diseño tomadas y los resultados obtenidos en cada etapa.

Ejercicio 1 – Chatbot con RAG

Tarea 1 - Creación de la base de datos vectorial

Metodología

Con el objetivo de facilitar la recuperación semántica de fragmentos relevantes, opté por implementar una base de datos vectorial utilizando herramientas ampliamente compatibles con el ecosistema LangChain. Se definieron tres componentes clave: motor vectorial, modelo de embeddings y estrategia de fragmentación textual.

Desarrollo / Implementación

Utilicé *Chroma* como motor de base de datos vectorial, ya que, al ser una de mis primeras experiencias en la creación de bases de datos vectoriales y al saber que ella es amigable para los desarrolladores por su capacidad de almacenar los documentos de texto, manejar la tokenización, vectorización e indexación automáticamente, consideré que es propicia para comenzar a acercarme de a poco al uso de herramientas de esta índole. Sobre el modelo de embedding, usé *distiluse-base-multilingual-cased-v1*, elegido por su capacidad de manejar textos multilingües con buena eficiencia semántica. Para la fragmentación de texto, reutilicé el enfoque basado en *spaCy*, dividiendo los documentos en bloques de tres oraciones. Finalmente, realicé una función denominada *búsqueda_semantica(query, k)* que, dado un input en lenguaje natural, devuelve los *k* fragmentos más similares utilizando búsqueda por similitud en *Chroma*.

Resultados

La función de búsqueda semántica permite recuperar fragmentos contextualmente relevantes desde la base vectorial, ofreciendo una interfaz sencilla y compatible con el flujo de trabajo de la arquitectura RAG. Esto sienta las bases para una integración eficiente del componente *retriever* con el sistema generativo del chatbot experto.

Tarea 2: Acceso y filtrado semántico de datos estadísticos

Metodología

Para permitir consultas precisas sobre el dataset estadístico del juego Pradera, enfoqué esta etapa en transformar la información estructurada en un formato interpretable por un modelo de lenguaje. El objetivo fue generar un sistema

capaz de identificar y aplicar filtros automáticamente a partir de una consulta en lenguaje natural.

Desarrollo / Implementación

Cargué el archivo .csv con Pandas y realicé un preprocesamiento que incluyó el descarte de estadísticas irrelevantes y la limpieza de espacios en blanco. Convertí el conjunto depurado en un diccionario clave-valor y construí un string (info_llm) con la información relevante, respetando las restricciones del enunciado de no pasar el dataset completo al modelo.

Para interpretar las consultas, utilicé el modelo *Mistral-7B-Instruct GGUF*, sobre el cual diseñé un prompt de tipo *few-shot* y lo encapsulé en una función llamada *extraer_filtro_estadistico*. Esta función se encarga de inferir, a partir de la consulta, el filtro que debe aplicarse sobre el dataset.

Finalmente, desarrollé dos funciones complementarias: *filtrar_set_estadistico*, que ejecuta el filtrado sobre el DataFrame, y *busqueda_tabular*, que integra todo el proceso desde la consulta hasta la devolución del resultado.

Resultados

Esta implementación permite transformar una pregunta en lenguaje natural en una búsqueda precisa dentro del set estadístico, sin comprometer rendimiento ni sobrecargar al modelo con datos innecesarios. La modularidad de las funciones facilita su reutilización e integración en futuras etapas del chatbot.

Tarea 3: Construcción de una base de datos de grafos y consultas en Cypher

Metodología

Para modelar las relaciones entre elementos del juego, construí una base de datos de grafos, dada su capacidad para representar conexiones complejas de manera natural y consultarlas mediante lenguajes como Cypher. Preferí este lenguaje porque al modelo LLM le resultaba más simple generar las consultas en ese lenguaje que en SQL. También decidí usar Neo4j por su claridad y facilidad de integración con Python y el notebook de colab. Como en tareas anteriores, el enfoque estuvo centrado en facilitar una interfaz intuitiva para que un modelo de lenguaje pueda generar la consulta correspondiente a partir de texto en lenguaje natural.

Desarrollo / Implementación

Partí del archivo .csv con relaciones entre entidades del juego, el cual cargué con Pandas para su limpieza y adaptación. Creé una instancia en Neo4j y establecí la conexión desde mi notebook de Colab, desde donde realicé la inserción de nodos y relaciones.

Para la interpretación de las consultas, reutilicé el modelo *Mistral-7B-Instruct* con otro prompt *few-shot* y encapsulé el proceso en la función *traducir_a_cypher*, la cual transforma una pregunta en lenguaje natural en una consulta Cypher válida. Finalmente, desarrollé la función llamada *busqueda_grafo*, que ejecuta este flujo completo y retorna los datos solicitados a partir de una consulta en lenguaje natural.

Resultados

La solución recupera exitosamente relaciones complejas del juego mediante lenguaje natural, delegando la generación de consultas Cypher al modelo LLM. Esta estrategia potencia la comprensión semántica del chatbot experto.

Tarea 4: Desarrollo de un clasificador de intención avanzado

Metodología

Para dirigir correctamente las consultas hacia la fuente de datos más adecuada (*Información*, *Estadísticas* o *Relaciones*), desarrollé y comparé dos clasificadores de intención: uno basado en un modelo entrenado previamente y otro utilizando un modelo de lenguaje con *few-shot prompting*. El objetivo fue identificar cuál ofrecía mayor precisión categorizando consultas en lenguaje natural.

Desarrollo / Implementación

Importé el modelo de redes neuronales desarrollado en el punto 6 del TP1P2 (modelo_NN.keras) y lo integré como primera opción clasificadora. Luego, desarrollé una función *clasificador_intencion_llm*, utilizando el mismo modelo *Mistral-7B-Instruct* empleado en tareas anteriores, configurado con ejemplos previos que reflejan correctamente el tipo de clasificación esperada.

Para evaluar ambos enfoques, armé un *test_set* de preguntas junto con su etiqueta correcta, simulando distintos tipos de intenciones (por ejemplo: ¿Cuál es la puntuación promedio del juego? → *Estadísticas*). Luego, ejecuté ambos modelos sobre ese conjunto para poder realizar las predicciones y medir el rendimiento basándome en la métrica *accuracy*.

Resultados

El clasificador basado en redes neuronales logró un *accuracy* del 70%, mientras que el modelo basado en LLM alcanzó un 85%, mostrando mayor capacidad de generalización y comprensión semántica. En base a este resultado, opté por utilizar el clasificador basado en LLM para integrarlo en el pipeline de decisiones del chatbot.

Tarea 5: Definición del pipeline de recuperación (retrieval)

Metodología

De acuerdo a lo pedido en el trabajo, se realizó una optimización del sistema de recuperación de información integrando mecanismos híbridos que combinen semántica y búsqueda clásica, así como mecanismos de re-ranqueo que mejoren la calidad de los resultados. Para esto, reutilicé mis funciones dinámicas para consultas sobre estructuras como grafos y tablas, cumpliendo con la restricción de no pasar estos objetos completos al contexto del modelo, y la función de búsqueda semántica, que la combiné con la sugerencia del profesor de utilizar 'BM25' como motor de búsqueda clásica.

Desarrollo / Implementación

Para consultas sobre la base de datos semántica, y como recién expresado en el apartado de metodología, combiné mi función *búsqueda_semantica* hecha anteriormente con un motor basado en BM25, un modelo clásico de recuperación por palabras clave. Fueron unificadas ambas estrategias en una nueva función llamada *retriever_hibrido*. Si bien esta combinación ofrecía buenos resultados, noté que el orden de relevancia no era óptimo, así que pasé rápidamente a la incorporación del proceso de ReRank.

Para el reranqueo, volví a utilizar el modelo *distiluse-base-multilingual-cased-v1* y desarrollé la función *rerank_resultados*, que reordena los chunks según la similitud semántica con la consulta. Finalmente, encapsulé todo en la función *buscar_fragmentos_hibrido_rerank*, que mejora sustancialmente la calidad de las respuestas.

En cuanto a las consultas tabulares y del grafo, reutilicé las funciones previamente desarrolladas (*busqueda_tabular* y *busqueda_grafo*), las cuales cumplen con el criterio de generar dinámicamente una query en base a la intención del usuario, sin exponer al modelo toda la estructura original.

Resultados

Este pipeline de recuperación permite al sistema responder de forma más precisa, eficiente y controlada. La búsqueda híbrida seguida de un reranqueo mejora mucho la relevancia de los fragmentos recuperados, mientras que las funciones dinámicas aseguran una interacción específica con datos tabulares y estructurados.

Tarea 6: Integración para generación y conversación

Metodología

Como tarea final del ejercicio 1, integré todos los componentes previamente desarrollados en un bucle conversacional funcional, con el objetivo de construir

un chatbot experto capaz de comprender consultas en lenguaje natural, recuperar información relevante desde distintas fuentes y generar respuestas útiles, manteniendo el contexto a lo largo del diálogo.

Desarrollo / Implementación

Utilicé nuevamente el modelo *Mistral-7B-Instruct* como generador, junto con los clasificadores y recuperadores desarrollados en tareas anteriores. Se define el siguiente flujo general del sistema:

- El usuario introduce una consulta libre.
- El sistema detecta el idioma (con langdetect) y clasifica la intención de la consulta (Información, Estadísticas o Relaciones).
- En función de la categoría, se activan recuperadores específicos (buscar_fragments_hibrido_rerank, busqueda_tabular o busqueda_grafo).
- Los fragmentos recuperados se condensan en un contexto y se generan respuestas utilizando el LLM.
- Si no se encuentra información útil, el sistema devuelve una respuesta genérica que sugiere reformular la consulta.
- Se mantiene un historial corto con las últimas cinco interacciones para preservar memoria conversacional.

También incorporé un traductor automático, usando deep-translator, para garantizar que las respuestas siempre se generen en el mismo idioma que el del usuario.

Resultados

La integración fue exitosa: el chatbot responde consultas de diversa índole con precisión, conserva la coherencia y respeta el idioma original del usuario. Se nota que tarda bastante en contestar dependiendo a qué base de datos consulta, y es una arista a mejorar. De todas formas, las respuestas generadas fueron relevantes, contextualizadas y claras. Una vez no pudo responder cuando sé que sí tenía la información disponible, pero si se le insiste y se reformula la pregunta logra resolver. Se comportó correctamente ante preguntas ambiguas, sugiriendo reformular e invitando a buscar en fuentes externas cuando consideraba que no tenía los datos precisados. Presentamos una conversación con el chat (se encuentra en el colab):

```
👋 Bienvenido al chatbot experto en Pradera. Escribí tu consulta o 'salir' para terminar.
Usuario: como gano el juego?
Asistente: El objetivo del juego de Meadow es mostrar mayor cantidad de puntos de victoria que tu oponente después de un número determinado de rondas. Los puntos se ganan principalmente mediante la colocación de tarjetas de colores en el tablero. Las reglas del juego son simples de aprender y el juego puede ser jugado en una variedad de niveles de complejidad. La primera persona en llegar a un número determinado
```


de puntos de victoria gana el juego. Para prepararse para el juego, debes armar algunos soportes para las tarjetas de cartón antes de comenzar.

Usuario: que ranking ocupa el juego?

Asistente: El juego de Meadow ocupa el puesto 209 en la clasificación de Valor de la estadística.

Usuario: quien diseño el juego?

Asistente: El juego de Meadow fue diseñado por Klemens Kalicki.

Usuario: que estrategias de juego puedo usar?

Asistente: En el juego de Meadow, las estrategias de juego se centran principalmente en la colocación de tarjetas de colores en el tablero para obtener puntos de victoria. Algunas estrategias comúnmente usadas incluyen:

- Intentar bloquear zonas importantes del tablero a tu oponente, especialmente en las rondas finales.

- Colocar tarjetas que aumenten el valor de otras tarjetas existentes en el tablero.

- Utilizar tarjetas especiales, como las que permiten intercambiar tarjetas, o las que dan bonus de puntos en determinadas situaciones.

- Intentar obtener el mayor número de puntos en las rondas finales, ya que el primer jugador en llegar a un número determinado de puntos de victoria gana el juego.

Recuerda que el juego de Meadow es un juego de estrategia y variabilidad, así que explorar diferentes estrategias y adaptarte a las acciones de tu oponente es

Usuario: cuantos nombres alternativos tiene meadow?

Asistente: El juego de Meadow no tiene nombres alternativos conocidos.

Usuario: salir

¡Hasta luego!

Ejercicio 2: Implementación de un agente autónomo ReAct con LangChain

Metodología

Para cerrar el pipeline de RAG convertí el flujo en un agente autónomo basado en el paradigma ReAct, según lo demandado en el enunciado del trabajo. Mi objetivo fue que el modelo no sólo genere lenguaje, sino que razone explícitamente sobre qué fuente de información consultar (documentos, tabla, grafo, Wikipedia o web) y ejecute esa acción para luego incorporar el resultado en su razonamiento. Para ello se definieron cinco herramientas (doc_search, table_search, graph_search, wikipedia_search y duckduckgo_search) que encapsulan cada uno de los sub-sistemas de recuperación, y diseñé un prompt de sistema altamente estructurado que fuerza al agente a usar la sintaxis Thought → Action → Action Input → Observation → ... → Final Answer sin salirse del formato.

Desarrollo / Implementación

1. Creación de Tools

- `doc_search(query)`: invoca el pipeline híbrido + re-rank sobre los documentos y sintetiza el texto.
- `table_search(query)`: filtra dinámicamente el DataFrame de estadísticas y formatea el resultado.
- `graph_search(query)`: genera y ejecuta consultas Cypher sobre Neo4j, extrayendo relaciones críticas (ej. diseñador, editorial).
- `wikipedia_search(query)`: utiliza WikipediaAPIWrapper para obtener definiciones o contexto general.
- `duckduckgo_search(query)`: recurre a DuckDuckGoSearchAPIWrapper para búsquedas web amplias.

2. Definición del agente en LangChain

- Instancié el LLM remoto mistralai/Mistral-7B-Instruct-v0.1 vía Together.ai con su API key. O
- Elegí `AgentType.CONVERSATIONAL_REACT_DESCRIPTION` (modo ReAct conversacional) y pasé un prefijo a modo de prompt que:
 1. Enumera las herramientas y su propósito.
 2. Impone la sintaxis paso a paso (Thought, Action, Action Input, Observation y Final Answer).
 3. Agrega reglas estrictas sobre idioma, veracidad, no repetición y uso de cada herramienta.

3. Función wrapper

- `agente_autonomo(pregunta)`: detecta el idioma de la consulta, invoca el agente con historial de conversación (vacío o limitado), traduce la respuesta si es necesario y la devuelve al usuario.

Resultados

El agente resultante es bastante aceptable:

- Razonó explícitamente antes de elegir cada herramienta, respetando el ReAct loop.
- Conectó de forma dinámica con las fuentes, sin exponer datos completos al LLM.
- En pruebas manuales, respondió correctamente a preguntas sobre reglas, estadísticas, relaciones, definiciones y búsquedas generales, cumpliendo el requerimiento de un agente autónomo integral.
- Es bastante veloz para contrastar información y responder.
- En muchas de las pruebas entró en loops pese a que el prompt ya era bastante estricto, y se tuvo que rigidizar aún más.
- Responde en el mismo idioma de la consulta.

Ejemplo:

```
agente_autonomo("en que se basa el juego?")
```

Respuesta del agente: (se omiten procesamientos intermedios del agente, ir al colab para verlo)

Respuesta: Pradera es un juego de cartas que se basa en el concepto de "gestión de manos", "redacción" y "colección establecida". Es un juego que se puede jugar solo en cualquier tienda de juegos porque tiene todos los elementos de un juego familiar. Hay un pequeño juego llamado Codex Naturalis que a menudo se conoce como "Minipradera" porque es similar a Pradera. Es posible que Codex Naturalis sea un juego similar a Pradera, pero no está claro si esto es cierto. Si tiene alguna pregunta sobre el juego, puede comunicarse con la tienda donde la compró o al distribuidor de juegos en su país.

Pradera es un juego de cartas que se basa en el concepto de "gestión de manos", "redacción" y "colección establecida". Es un juego que se puede jugar solo en cualquier tienda de juegos porque tiene todos los elementos de un juego familiar. Hay un pequeño juego llamado Codex Naturalis que a menudo se conoce como "Minipradera" porque es similar a Pradera. Es posible que Codex Naturalis sea un juego similar a Pradera, pero no está claro si esto es cierto. Si tiene alguna pregunta sobre el juego, puede comunicarse con la tienda donde la compró o al distribuidor de juegos en su país.

Conclusión

He construido un sistema RAG completo y un agente ReAct que integra recuperación vectorial, tabular, gráfica y búsquedas en línea. El clasificador LLM supera al entrenado con redes neuronales (85 % vs. 70 % de accuracy) y el pipeline híbrido con rerank mejora la relevancia de los resultados ampliamente por sobre la búsqueda híbrida por sí sola. Probé el agente en varios idiomas y responde correctamente, aunque ninguna consulta baja de 30 segundos de latencia debido al procesamiento distribuido.

Se cumplieron todos los objetivos: se definieron embeddings multilingües, fragmentación, filtros dinámicos, funciones independientes y un prompt ReAct preciso. El agente elige la herramienta adecuada y mantiene un contexto multilingüe.

Para futuro trabajo, debería considerar:

1. Integrar métricas automáticas de evaluación de respuestas.
2. Explorar memorias a largo plazo para personalizar el diálogo.
3. Recoger feedback para refinar prompts y filtros.
4. Optimizar la infraestructura para reducir los tiempos de adquisición de respuesta.



Tecnicatura Universitaria en Inteligencia Artificial
Facultad de Ciencias Exactas, Ingeniería y
Agrimensura
Universidad Nacional de Rosario

5. Usar un modelo más robusto, que no necesite reglas tan exigente para cumplir con las tareas. A mi agente le cuesta bastante decir que no tiene una respuesta y pedir una reformulación, pese a que fue explicitado varias veces que lo haga.